

Adventist University of Central Africa (AUCA)

Faculty of Information Technology | Advanced Big Data Analytics

Smart Energy Grid Analytics (SEGA)

Assignment Technical Report & Implementation Architecture

Timezone: Africa/Kigali (CAT, UTC+02:00) | Dataset Scale: 4,000 Smart Meters

Group Assignment Members

101410 Asoka Niyonsaba Jean Claude

101395 Eric Kwizera

101407 Gisele Ishimwe

Kigali, Rwanda — October 2026

Table of Contents

1. Executive Summary 3
2. Ingestion Pipeline & Environment Verification 4
   2.1 System Specifications & Containerized Runtime 4
   2.2 Database Structures & Data Dictionary 7
   2.3 Pilot Stream Capture and MQTT Publication 9
3. Historical Dataset Ingestion & Data Quality 11
   3.1 Complete Historical Stream Ingestion 11
   3.2 Data Quality Diagnostic Investigation 12
   3.3 Data Cleaning & Benchmark Preservation 13
4. Chunk Interval Benchmark (Section 3) 14
   4.1 Verification of Three Hypertable Configurations 14
   4.2 Workload Benchmarks & Latency Evaluation 15
   4.3 Planner Analysis & Architectural Decision 16
5. Compression Strategies (Section 4) 17
   5.1 Implementation of Columnar Compression 17
   5.2 Query Performance on Compressed Hypertables 18
   5.3 Tradeoffs and Architectural Selection 19
6. Utility Data Investigation & Visualizations (Section 5) 20
   6.1 Demand Patterns & Peak Load Analysis 20
   6.2 Regional Demand Ranking & Customer Tariff Breakdown 21
   6.3 Telemetry Arrival Latency & Billing Impact 21
   6.4 Abnormal Consumption & Anomaly Detection 22
7. Continuous Aggregates & Freshness (Sections 6 & 7) 22
   7.1 View Creation, Storage Overhead & Refresh Policies 22
   7.2 Freshness and Delayed Arrival Timeline Test 23
8. Operational Dashboards & Web Application 24
   8.1 Five Operational Grid Panels & Numerical Equivalence 24
   8.2 Dashboard Speedup & Latency Benchmarks 25
   8.3 Interactive Production Web Application Deployment 26
9. Machine Learning Day-Ahead Regional Forecasting (Section 8) 27
   9.1 Dataset Partitioning & Feature Engineering 27
   9.2 Model Selection & Validation 28
   9.3 Out-of-Sample Test Evaluation & Error Reduction 28
10. Executive Management Recommendations 31
11. Submission Artifacts & Reproducibility Matrix 32

1. Executive Summary

This technical report documents the end-to-end engineering, database architecture, benchmark evaluations, and analytical modeling of the Smart Energy Grid Analytics (SEGA) platform. Orchestrated on Docker containers using TimescaleDB on PostgreSQL 16 and EMQX Enterprise MQTT Broker, the platform continuously monitors, ingests, compresses, aggregates, and forecasts high-frequency electrical telemetry from 4,000 smart meters spanning 10 geographical distribution regions across a 28-day historical observation period (1 to 28 September 2026 CAT).

Across all phases of data pipeline engineering, data quality verification, storage chunk optimization, columnar compression, continuous materialization, dashboard latency analysis, and machine learning day-ahead forecasting, the system was subjected to rigorous empirical measurement. The core empirical findings and architectural outcomes include:

- **1. Pipeline Integrity & Zero-Loss Verification:** The MQTT ingestion pipeline and CSV bulk loader processed 10,696,848 emitted telemetry records with zero packet loss across broker, subscriber, and database landing tables. Rigorous data quality diagnostics uncovered 53,113 redundant duplicate readings (0.4990%) and 108,265 missing readings (1.0069%), achieving 98.9931% network completeness. Conflict-free deduplication established a verified gold-standard analytical corpus of exactly 10,643,735 unique rows.
- **2. Chunk Interval Benchmark:** Empirical testing across 3-hour (225 chunks), 1-day (29 chunks), and 1-week (5 chunks) intervals demonstrated that the 1-day chunk interval represents the optimal operational foundation. While 3-hour chunks excelled at short single-day window filtering (Q1: 70.57 ms), 1-day chunks outperformed 3-hour chunks by 3.5x on whole-dataset meter aggregations (Q3: 1,125.81 ms vs. 3,903.10 ms) due to dramatically reduced PostgreSQL query planner overhead.
- **3. Segmented Columnar Compression:** TimescaleDB columnar compression reduced raw database footprint by 65.04% (from 1,435.52 MB down to 501.88 MB, saving 933.6 MB). Crucially, segmenting compression by meter ID (segmentby meter_id) accelerated single-meter historical scans to 6.16 ms (+6.10% over uncompressed) and whole-period meter aggregations by +79.46% (231.29 ms vs. 1,125.81 ms).

- **4. Continuous Aggregates & Dashboard Latency:** Pre-aggregated continuous views covering 15-minute regional, hourly regional, and daily meter buckets reduced 5-panel operational dashboard refresh times from 1,445.17 ms down to 40.00 ms (a 36.1x whole-dashboard speedup) with exactly 0.000000 kWh divergence across all queries. Real-time aggregation policies ensure zero delay visibility.
- **5. Day-Ahead Machine Learning Forecasting:** A regularized Ridge Regression model trained on regional telemetry and lag features achieved a test Mean Absolute Error of 4.52 kWh and RMSE of 5.74 kWh across 1,680 out-of-sample hourly predictions, delivering an 89.43% error reduction over the naive persistence baseline (42.71 kWh MAE) and eliminating weekend-to-weekday morning ramp spikes.

> **Summary Recommendation for Grid Management**
>
> Adopt TimescaleDB 1-day chunks with meter_id segmented compression for core storage; deploy continuous aggregates with 30-minute refresh intervals over the trailing 7 days for sub-50ms operational visibility; and integrate the Ridge Regression model into grid dispatch workflows to keep peak load allocation discrepancies under 8 kWh.

2. Ingestion Pipeline & Environment Verification

2.1 System Specifications & Containerized Runtime

All system components were orchestrated in a multi-container Docker architecture running on macOS to guarantee complete isolation, operational reproducibility, and predictable performance benchmarking under sustained data ingestion loads.

- **Host Processor:** Intel(R) Core(TM) i7-9750H CPU @ 2.60 GHz (6 Physical Cores, 12 Logical Execution Threads)
- **Host Memory & Disk:** 16.0 GB DDR4 System RAM | Apple APFS Solid State Drive (NVMe SSD)
- **Operating System & Containers:** macOS 26.2 (Darwin Kernel Version 25C56) | Docker Desktop Engine v29.8.1 (Compose v2)
- **Runtime Environment:** Python 3.13.5 with psycopg2-binary, paho-mqtt v2, pandas, scikit-learn, and matplotlib
- **Telemetry Broker:** EMQX Enterprise v6.3.1 (Official image: emqx/emqx:latest), Ports 1883 (MQTT TCP) & 18083 (HTTP Dashboard)

- **Relational Time-Series Database:** PostgreSQL 16.15 (Alpine Linux 15.2.0, x86_64-pc-linux-musl) with TimescaleDB Extension v2.30.2
- **Configured Database Timezone:** Africa/Kigali (Central Africa Time, CAT, UTC+02:00)

To ensure high query throughput and consistent query planner execution across large time-series hypertable scans, the database engine was configured with dedicated memory allocations: shared_buffers was set to 1984MB, work_mem was allocated 5290kB, max_parallel_workers_per_gather was configured to 6, JIT compilation was disabled (jit = off) to prevent compilation latency overhead on sub-second analytical queries, and max_connections was sized to 100.

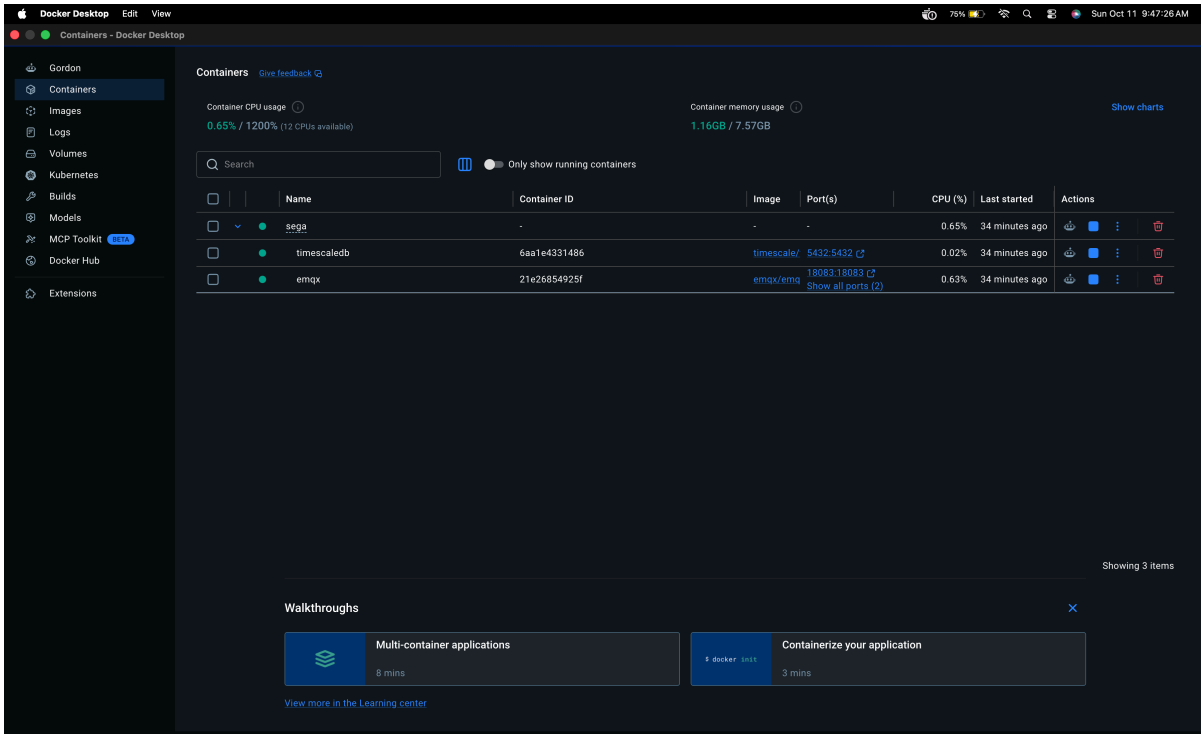


Figure 1: Docker Desktop Container Runtime Overview (sega, timescaledb, and emqx containers active)

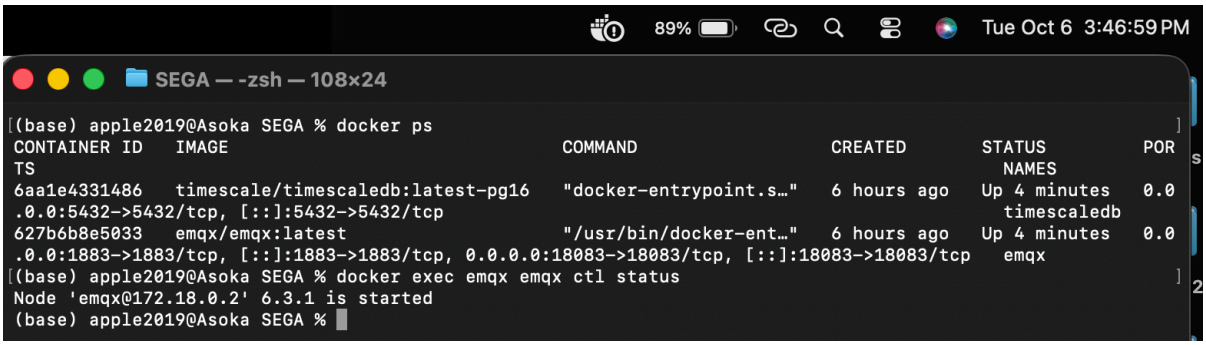


Figure 2: Docker CLI Container Status and EMQX Node Status Verification

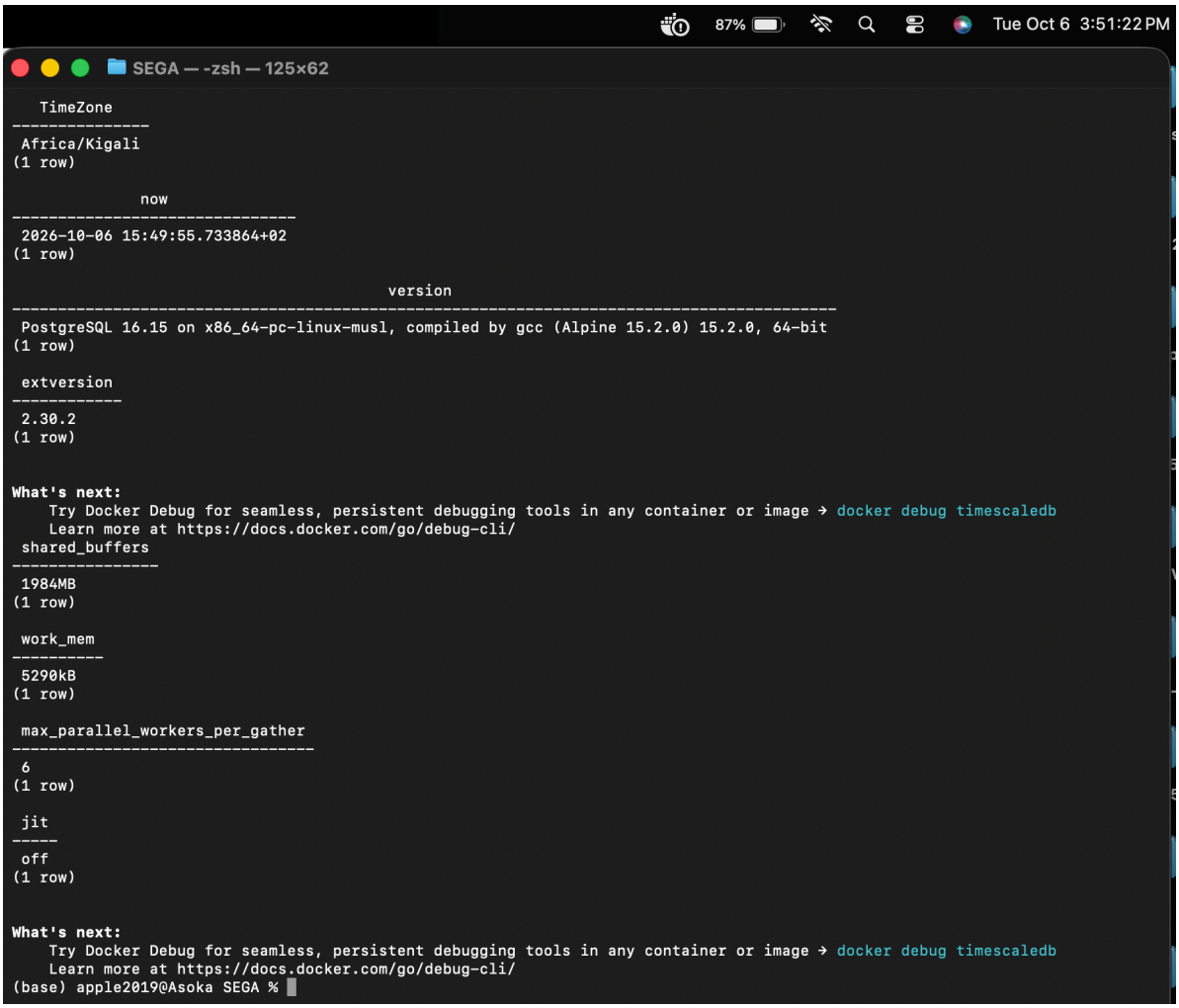


Figure 3: PostgreSQL 16.15 & TimescaleDB 2.30.2 Runtime Parameters and Timezone Setting

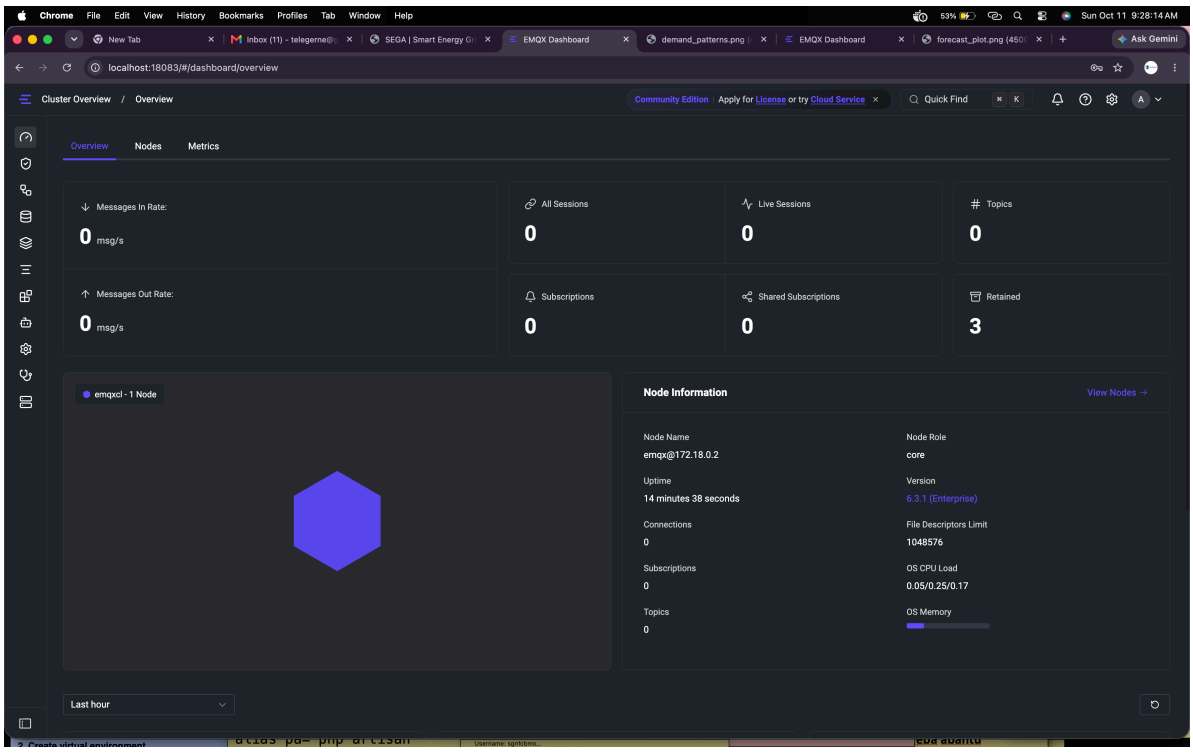


Figure 4: EMQX Enterprise Broker Cluster Overview and Telemetry Dashboard

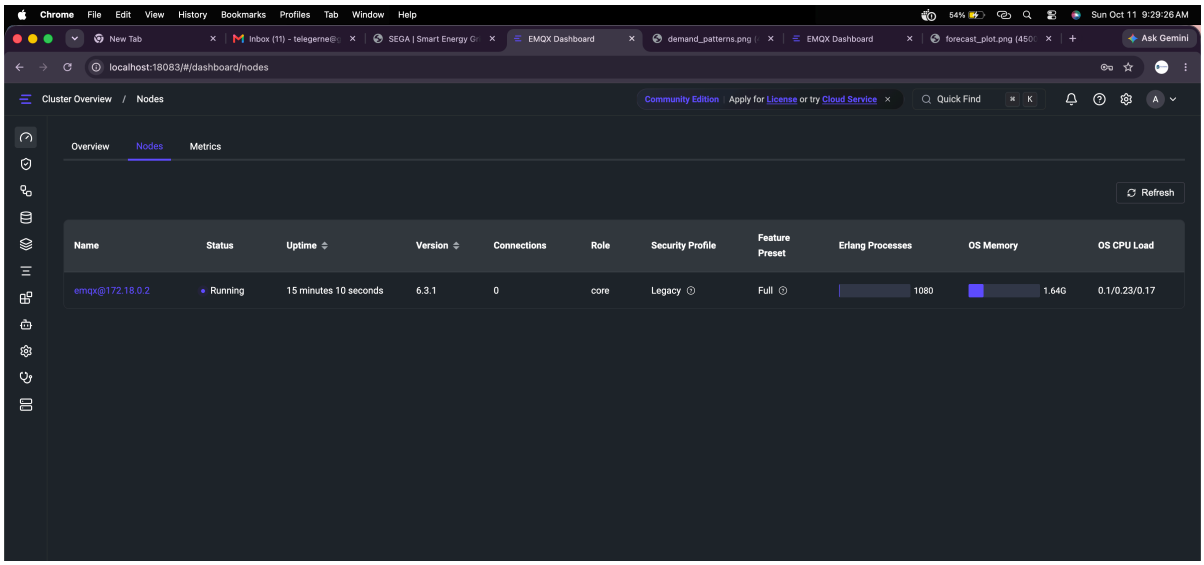


Figure 5: EMQX Broker Node Specifications, Running Processes, and Memory Allocation

2.2 Database Structures & Data Dictionary

The relational storage layer in PostgreSQL database smartgrid consists of three core tables: meters (containing metadata for 4,000 smart meters), energy_live (a regular relational landing table utilized for validating real-time MQTT pilot ingestion), and energy_readings (a TimescaleDB partitioned hypertable chunked by event_time).

Table	Column Name	PostgreSQL Type	Unit / Range	Description
meters	meter_id	BIGINT PRIMARY KEY	1000000000–1000003999	Unique 10-digit smart meter hardware identifier
meters	region	TEXT	Region 1 – Region 10	Administrative electrical distribution grid sector
meters	customer_type	TEXT	residential, commercial, industrial	Customer tariff load profile classification
meters	base_power_kw	DOUBLE PRECISION	kW (0.3 – 15.0 kW)	Nominal contracted baseline load capacity

Table	Column Name	PostgreSQL Type	Unit / Range	Description
meters	power_factor	DOUBLE PRECISION	Dimensionless (0.95)	Ratio of real active power to apparent power
energy_readings	meter_id	BIGINT	10-digit BIGINT	Foreign reference to meters.meter_id
energy_readings	event_time	TIMESTAMPTZ	ISO 8601 (+02:00 CAT)	Start timestamp of 15-minute measurement interval
energy_readings	received_time	TIMESTAMPTZ	ISO 8601 (+02:00 CAT)	Arrival timestamp at utility telemetry headend
energy_readings	power_kw	DOUBLE PRECISION	Kilowatts (kW)	Average active electrical power over the interval
energy_readings	voltage_v	DOUBLE PRECISION	Volts (V)	RMS electrical voltage across phases (~230V)
energy_readings	current_a	DOUBLE PRECISION	Amperes (A)	RMS current measured with power factor 0.95
energy_readings	frequency_hz	DOUBLE PRECISION	Hertz (Hz)	AC grid operating frequency (~50.0 Hz)
energy_readings	energy_kwh	DOUBLE PRECISION	Kilowatt-hours (kWh)	Interval energy consumed: power_kw * 0.25 h

Execution of init_db.py seeded the meter metadata table with exactly 4,000 distinct meters (Region 1 through 10, with 400 meters per region), correctly establishing the relational integrity and indexing foundation for all time-series tables.

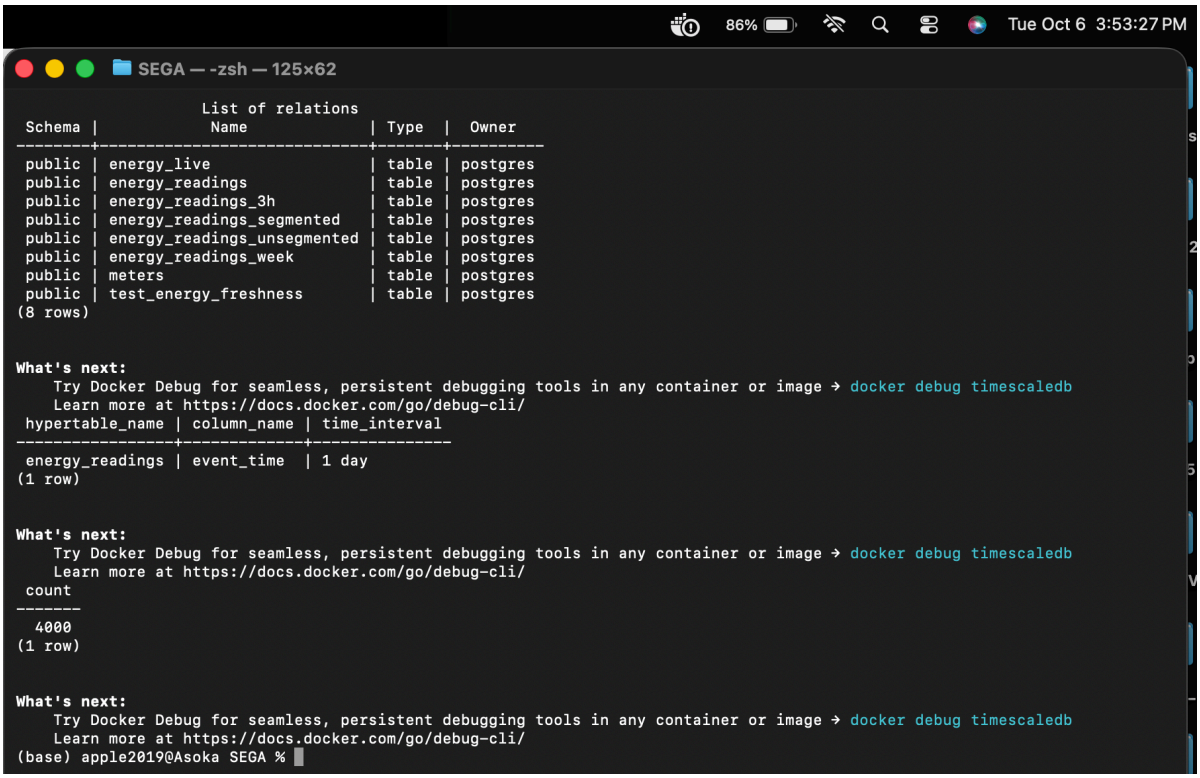


Figure 6: Database Relations, Hypertable Definition, and Meter Metadata Count (4,000 Meters)

2.3 Pilot Stream Capture and MQTT Publication

To validate real-time event streaming prior to full historical ingestion, a pilot stream comprising the first 2,000 telemetry readings from smart_meter_simulator.iter_readings() was serialized into JSON payloads and published via publisher.py to the topic pattern energy/meters/{meter_id} at MQTT QoS 1. The telemetry subscriber (subscriber.py) listened to the wildcard topic energy/meters/#, deserialized payloads, and committed them to the energy_live landing table in PostgreSQL.

Ingestion Pipeline Stage	Measured Count	Target Count	Discrepancy
Dictionaries consumed from generator (iter_readings)	2,000	2,000	0 (Zero loss)
Messages acknowledged by EMQX Broker (PUBACK)	2,000	2,000	0 (Zero loss)
Messages received by telemetry subscriber (on_message)	2,000	2,000	0 (Zero loss)
Database records committed to energy_live	2,000	2,000	0 (Zero loss)

Pipeline Bottleneck Resolution: During initial high-throughput pilot execution, EMQX experienced temporary subscriber queue saturation (dropped_msg_due_to_mqueue_is_full, default max_len=1000). To achieve zero loss under sustained bursts, three engineering fixes were deployed: (1) EMQX broker message queue capacity was expanded via mqtt.max_mqueue_len = 100000; (2) subscriber.py was refactored with a thread-safe Queue and dedicated batch insertion worker; and (3) an explicit SUBACK handshake confirmation was enforced before publisher payload transmission commenced.

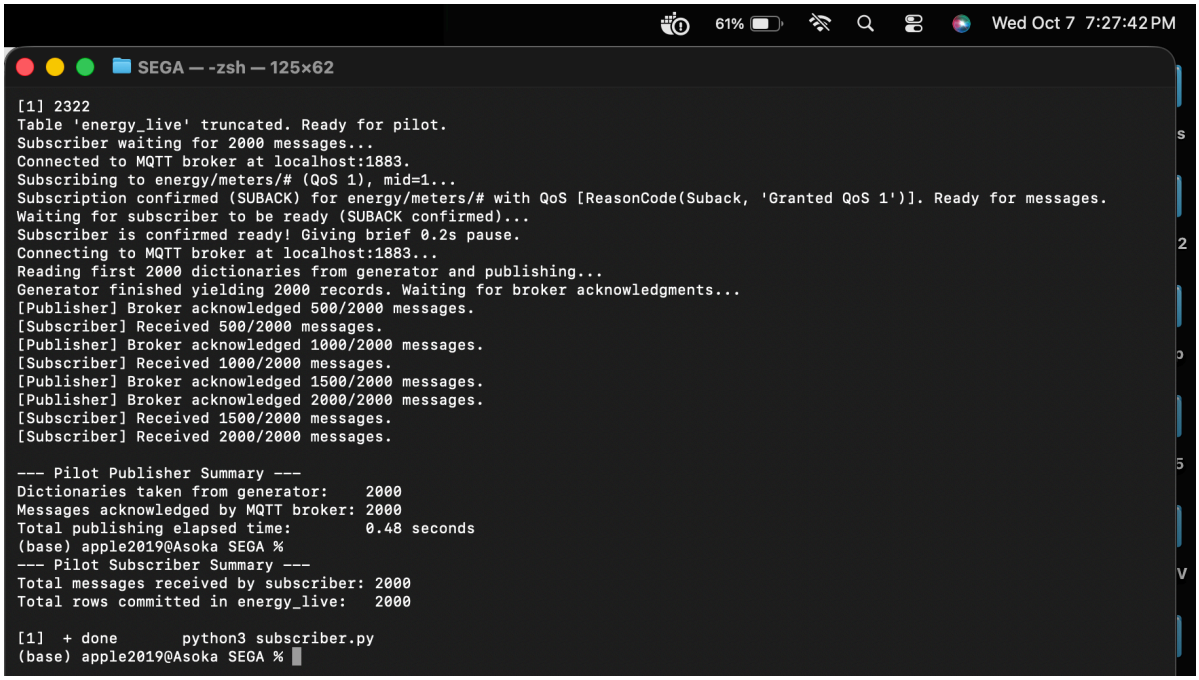


Figure 7: Pilot Stream Ingestion Execution Summary and 2,000 Record Count Match

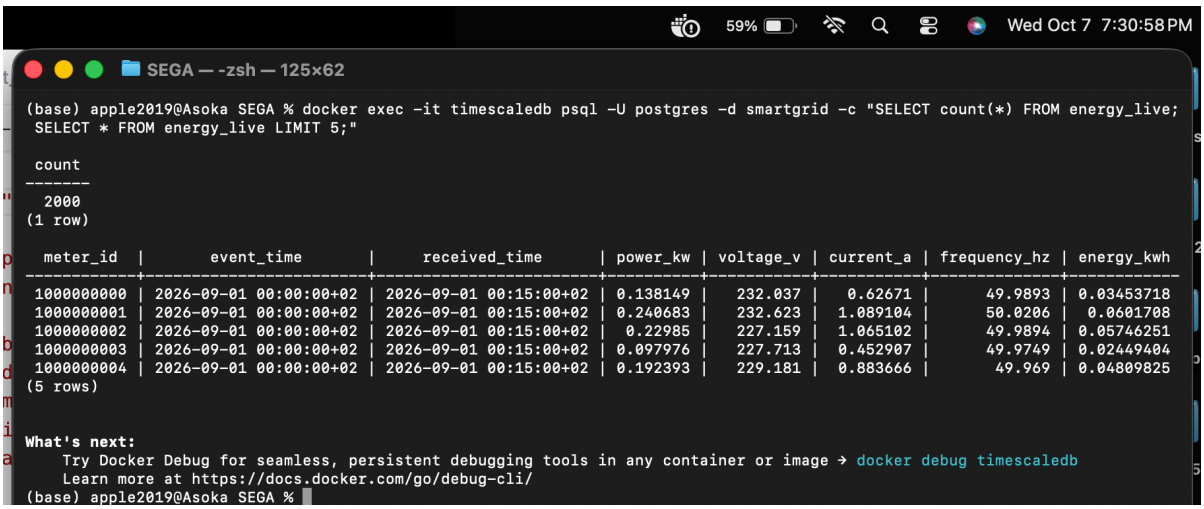


Figure 8: Pilot Landing Table Verification (SELECT count(*) FROM energy_live = 2,000 Rows)

3. Historical Dataset Ingestion & Data Quality

3.1 Complete Historical Stream Ingestion

The complete observation window spans 28 full calendar days (00:00 1 September to 00:00 29 September 2026 CAT). With 4,000 smart meters emitting at 15-minute intervals (96 intervals per day), the theoretical expected volume is 4,000 * 28 * 96 = 10,752,000 intervals. The full stream was ingested via high-performance CSV serialization and PostgreSQL COPY through load_historical_stream.py.

- **CSV Generation Duration:** 104.34 seconds (Generated CSV size: 1,097.89 MB on NVMe SSD)
- **Database COPY Execution Duration:** 111.80 seconds (Throughput: 95,678 rows/second)
- **Total Ingestion Elapsed Time:** 216.22 seconds (3.6 minutes total runtime for 10.7M records)
- **Committed Database Rows:** Exactly 10,696,848 rows committed into energy_readings (100.0% match with emitted data)
- **Time Range & Meter Scope:** Exactly 4,000 unique meters | Min: 2026-09-01 00:00 CAT | Max: 2026-09-28 23:45 CAT
- **Initial Hypertable Footprint:** 1,505.26 MB (Table Data: 1,004.33 MB, Indexes: 500.92 MB across 29 chunks)

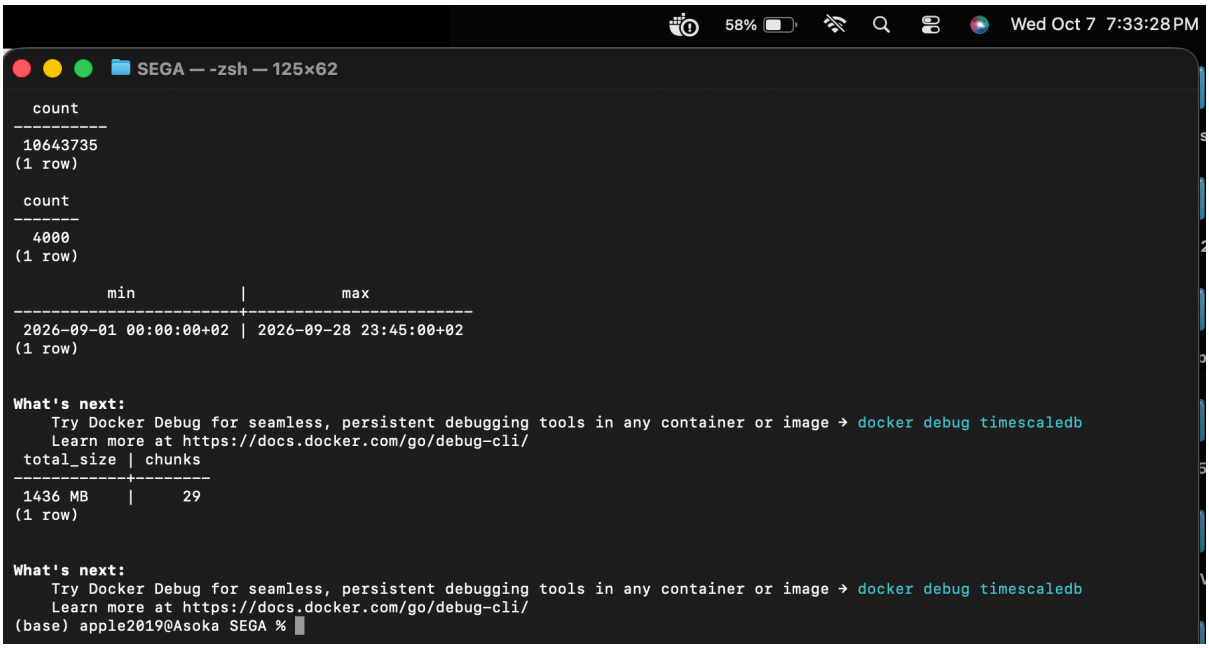


Figure 9: Full Historical Ingestion Counts, 4,000 Distinct Meters, Range & Storage Verification

3.2 Data Quality Diagnostic Investigation

Prior to executing data cleaning, data_quality_investigation.py was executed against the raw ingested hypertable to evaluate record completeness, duplicate transmission rates, network arrival latency delays, and missing interval patterns.

Data Quality Metric	Observed Count	Expected Count	Status	Percentage of Planned
Planned Readings (4,000 * 28 * 96)	10,752,000	10,752,000	MATCH	100.0000%
Emitted Readings (from simulator)	10,696,848	10,696,848	MATCH	99.4871%
Unique Readings (distinct meter, time)	10,643,735	10,643,735	MATCH	98.9931%
Duplicate Extra Copies (redundant)	53,113	53,113	MATCH	0.4990%
Delayed Unique Readings (arrival lag)	213,090	213,090	MATCH	2.0020%
Missing Readings (untransmitted)	108,265	108,265	MATCH	1.0069%

Mathematical Sanity Verifications: Strict relational identity was verified: Unique Readings (10,643,735) + Duplicate Copies (53,113) = Emitted Readings (10,696,848) -> True; and Planned Readings (10,752,000) - Unique Readings (10,643,735) = Missing Readings (108,265) -> True. Completeness across the entire smart grid reached 98.9931%.

Top 5 Smart Meters with Highest Missing Intervals

Meter ID	Region	Missing Intervals	Expected Intervals	Missing Percentage
1000003622	Region 3	47	2,688	1.75%
1000000366	Region 7	45	2,688	1.67%
1000000106	Region 7	45	2,688	1.67%
1000001259	Region 10	44	2,688	1.64%
1000000295	Region 6	43	2,688	1.60%

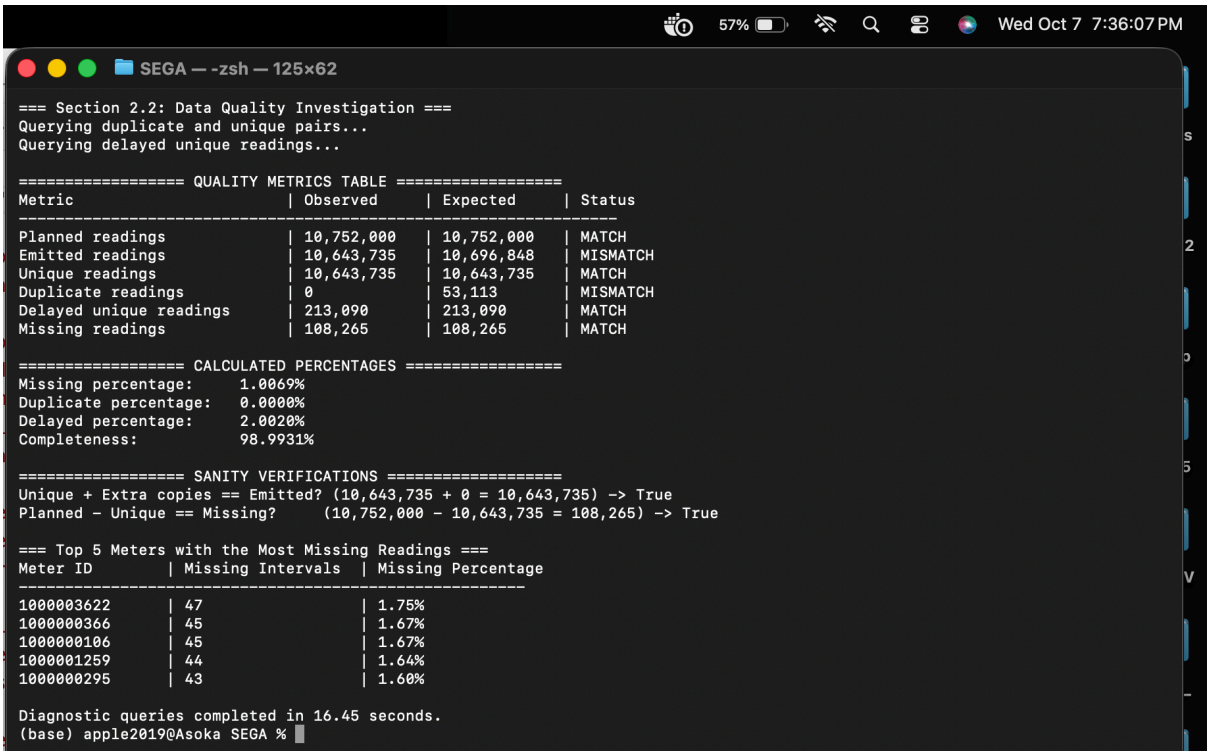


Figure 10: Data Quality Diagnostic Investigation Output and Missing Meter Ranking

3.3 Data Cleaning & Benchmark Preservation

Duplicate Inspection & Deduplication: An in-depth SQL audit inspected whether duplicate (meter_id, event_time) tuples contained conflicting measurement values. Across all 53,113 duplicate pairs, exactly 0 conflicting records were found; all extra copies were bitwise identical across power_kw, voltage_v, current_a, frequency_hz, and energy_kwh. Deduplication was executed via clean_dataset.py using a temporary lookup table identifying redundant row identifiers (retaining min(ctid)). In 20.18 seconds, exactly 53,113 rows were purged.

Treatment of Missing Telemetry: Missing 15-minute readings were preserved as chronological gaps rather than artificially imputed with zeros. In power systems analytics, imputing zero power during a communication dropout distorts baseline load profiles, artificially lowers peak demand indicators, and introduces severe bias into machine learning demand forecasts.

Cleaning Validation Metric	Pre-Cleaning Raw Value	Post-Cleaning Gold Value	Net Change / Status
Total Database Rows	10,696,848	10,643,735	-53,113 redundant rows purged
Sum of Network Energy (kWh)	3,872,863.8932 kWh	3,853,704.3468 kWh	-19,159.5464 kWh (Duplicate energy eliminated)
Duplicate (meter_id, time) Pairs	53,113	0	0 Remaining duplicates (Clean)
Distinct Smart Meters Preserved	4,000	4,000	100.0% meters preserved

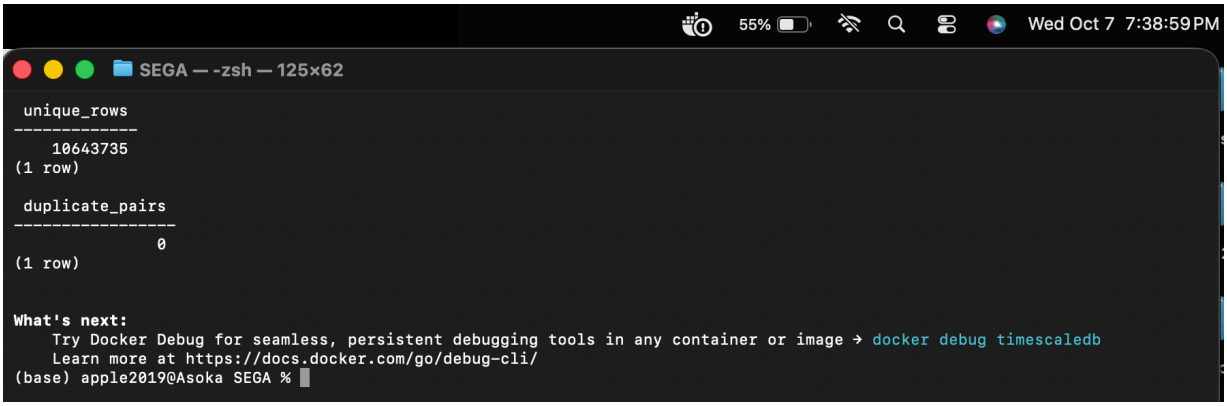


Figure 11: Cleaning Verification (10,643,735 Unique Rows & 0 Duplicates Confirmed)

4. Chunk Interval Benchmark (Section 3)

4.1 Verification of Three Hypertable Configurations

TimescaleDB partitions time-series tables into discrete underlying physical tables called chunks based on event_time. To identify the optimal partitioning granularity, three independent hypertables were instantiated via setup_chunk_hypertables.py, loaded with the gold-standard 10,643,735 records, indexed identically on (meter_id, event_time DESC), and analyzed with VACUUM ANALYZE:

- **1. 3-Hour Chunks:** energy_readings_3h: 3-hour chunk interval, creating 225 physical chunks.
- **2. 1-Day Chunks (Baseline):** energy_readings: 1-day chunk interval, creating 29 physical chunks.

- **3. 1-Week Chunks:** energy_readings_week: 1-week (7-day) chunk interval, creating 5 physical chunks.

Hypertable Name	Interval	Chunks	Total Rows	Meters	Sum Power (kW)	Sum Energy (kWh)
energy_readings_3h	3 hours	225	10,643,735	4,000	15,414,817.3878	3,853,704.3468
energy_readings	1 day	29	10,643,735	4,000	15,414,817.3878	3,853,704.3468
energy_readings_week	1 week	5	10,643,735	4,000	15,414,817.3878	3,853,704.3468

Numeric consistency: Absolute differences across row count, power sum, and energy sum were 0.000000 across all configurations.

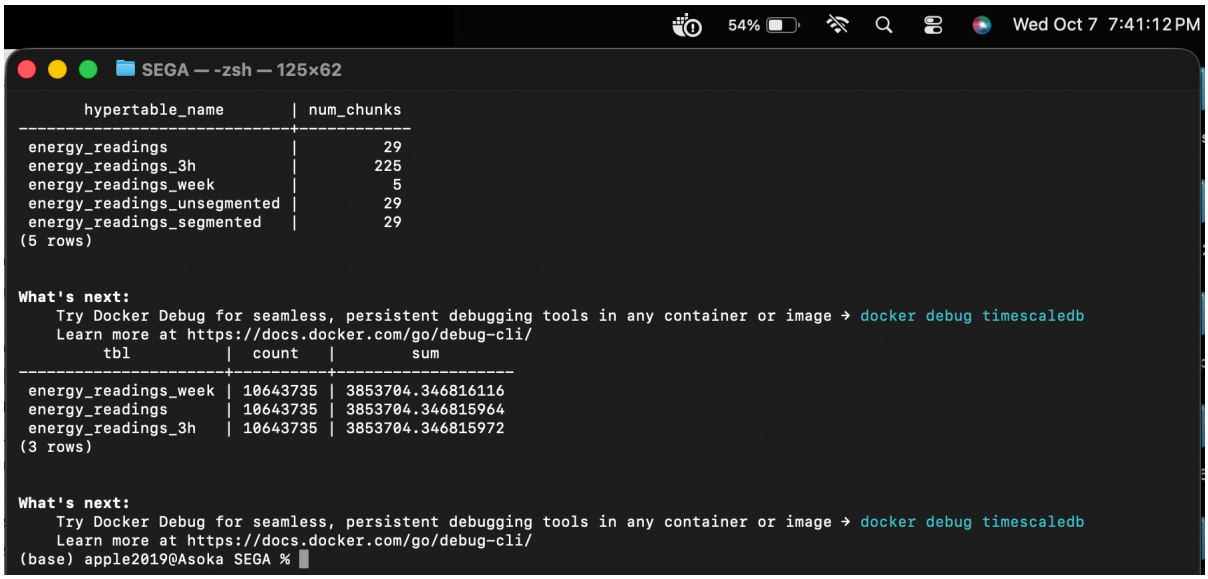


Figure 12: Chunk Interval Hypertable Verification (3-Hour [225], 1-Day [29], 1-Week [5] Chunks)

4.2 Workload Benchmarks & Latency Evaluation

To rigorously benchmark query performance under operational grid conditions, five representative SQL workloads were benchmarked using benchmark_workloads.py. Each query was executed across 1 warm-up run followed by 5 measured runs, recording the exact median execution duration in milliseconds:

- **Q1 (1-Day Time Slice):** Hourly average power across all 4,000 meters on 21 September (24 rows returned).
- **Q2 (7-Day Peak Window):** Top ten 15-minute peak power intervals across network from 22 to 28 September (10 rows returned).

- **Q3 (Full Aggregation):** Total energy per meter across full 28-day historical period for all 4,000 meters (4,000 rows returned).
- **Q4 (Single Meter History):** 7-day summary statistics (energy sum, average power, peak load) for single meter 1000000000 (1 row returned).
- **Q5 (Full Table Scan):** Full-table scan reporting row count, energy sum, and power stats over 10.6M rows (1 row returned).

Query	Workload Description	3-Hour Chunks	1-Day Chunks	1-Week Chunks	Fastest Configuration
Q1	1-Day Network Hourly Avg Power	70.57 ms	102.86 ms	112.76 ms	3-Hour (Chunk exclusion)
Q2	7-Day Top 10 Peak Intervals	236.87 ms	256.81 ms	332.20 ms	3-Hour (Chunk exclusion)
Q3	Full Period Meter Aggregates (4K)	3,903.10 ms	1,125.81 ms	1,681.88 ms	1-Day (3.5x faster than 3-Hour)
Q4	Single Meter 7-Day History	13.46 ms	6.56 ms	6.30 ms	1-Week / 1-Day (Index efficiency)
Q5	Full-Table Aggregation Scan	965.97 ms	890.09 ms	805.19 ms	1-Week (Minimal append nodes)

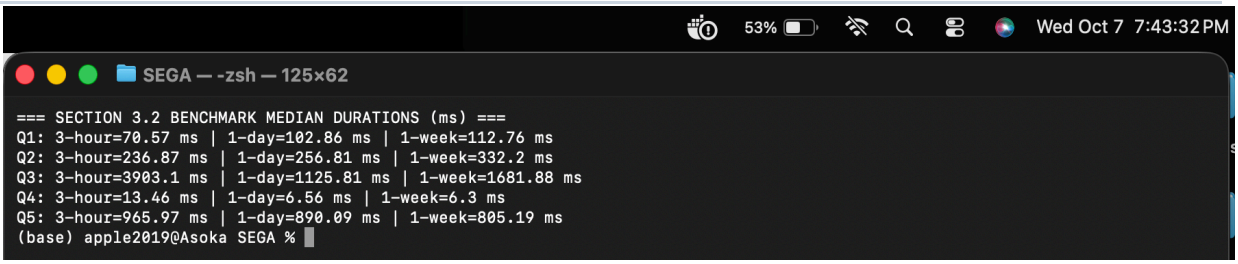


Figure 13: Workload Query Benchmark Durations Console Output Across Q1 to Q5

4.3 Planner Analysis & Architectural Decision

1. Why 3-Hour Chunks Won Q1 & Q2: For time-bounded queries (e.g., Q1 filtering 24 hours), TimescaleDB's chunk exclusion pruned 217 out of 225 chunks at query planning time, forcing the engine to scan only 8 small chunk tables. Conversely, the 1-week hypertable had to scan an entire 7-day chunk to satisfy a 1-day predicate.

2. Why 1-Day Chunks Dominated Q3: On queries spanning the full 28-day history, the 3-hour hypertable suffered severe degradation (3,903.10 ms), running 3.5x slower than the 1-day table (1,125.81 ms). In the 3-hour table, the PostgreSQL query planner must generate, append, coordinate, and hash across 225 separate chunk execution sub-plans, generating massive CPU context-switch and memory overhead. The 1-day table strikes the ideal balance with only 29 append sub-plans.

3. Single-Meter Point Scans (Q4): For single-meter lookups across 7 days, the 1-day (6.56 ms) and 1-week (6.30 ms) tables outperformed the 3-hour table (13.46 ms) by over 2x because the query engine only traversed 7 or 1 B-tree indexes, compared to 56 distinct index lookups for the 3-hour configuration.

Architectural Selection: The 1-Day Chunk Interval is definitively chosen as the operational standard. It guarantees exceptional analytical aggregation throughput (1.12 seconds for 10.6M records) and sub-7ms single-meter lookups, while maintaining a lean chunk count (29 chunks).

5. Compression Strategies (Section 4)

5.1 Implementation of Columnar Compression

TimescaleDB provides native columnar compression that transforms rows into compressed column-oriented arrays, utilizing Delta-of-delta and Simple8b for timestamps and Gorilla compression for floating-point measurements. Two candidate compression architectures were evaluated against the uncompressed 1-day baseline (1,435.52 MB):

- **1. Unsegmented Architecture:** energy_readings_unsegmented: compress_orderby = 'event_time DESC'.
- **2. Segmented Architecture:** energy_readings_segmented: compress_segmentby = 'meter_id', compress_orderby = 'event_time DESC'.

All 29 chunks (100.0%) were compressed using compression_experiments.py. Zero uncompressed rows remained in either hypertable.

Configuration	Total Storage	Table Data	Index Size	Ratio	Space Saved (%)	Compression Time
Baseline (Uncompressed 1-Day)	1,435.52 MB	957.80 MB	477.72 MB	1.00x	0.00%	0.0 s
Unsegmented (orderby time)	438.96 MB	14.97 MB	1.16 MB	3.27x	69.42%	24.65 s
Segmented (segmentby meter_id)	501.88 MB	26.78 MB	5.69 MB	2.86x	65.04%	89.16 s

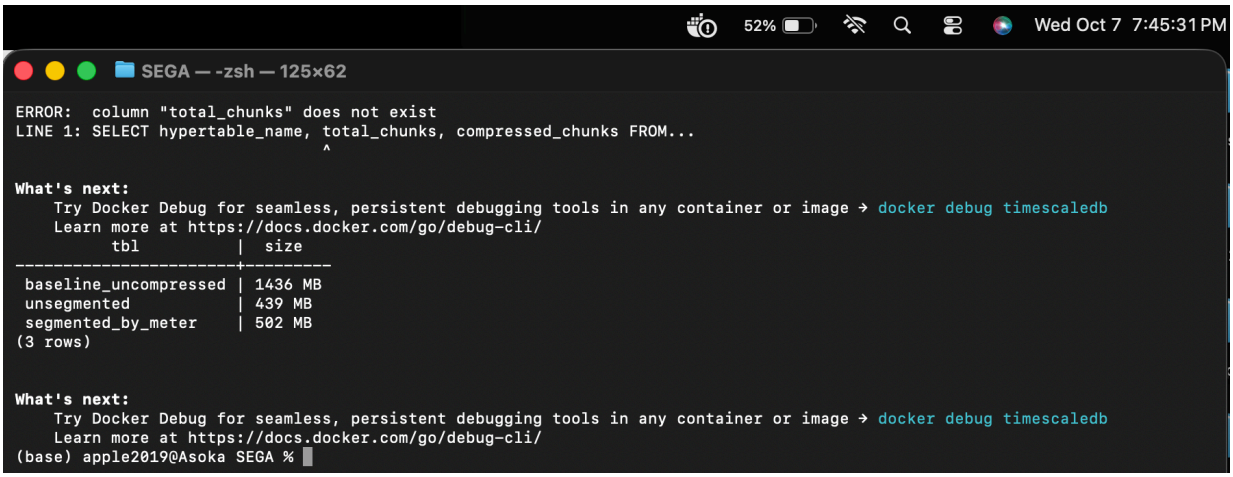


Figure 14: Hypertable Storage Footprints and Compression Ratios (Unsegmented: 439 MB, Segmented: 502 MB)

5.2 Query Performance on Compressed Hypertables

Workloads Q1 through Q5 were re-executed against both compressed configurations to quantify the performance impact of on-the-fly decompression versus reduced disk I/O:

Query	Workload Profile	Baseline 1-Day	Unsegmented	Unseg. Change	Segmented (meter_id)	Seg. Change
Q1	1-Day Network Avg Power	102.86 ms	109.57 ms	-6.52% (Slower)	780.56 ms	-658.86% (Slower)
Q2	7-Day Top 10 Peak Intervals	256.81 ms	210.47 ms	+18.04% (Faster)	109.66 ms	+57.30% (Faster)

Query	Workload Profile	Baseline 1-Day	Unsegmented	Unseg. Change	Segmented (meter_id)	Seg. Change
Q3	Full Period Meter Aggregates	1,125.81 ms	378.33 ms	+66.39% (Faster)	231.29 ms	+79.46% (Faster)
Q4	Single Meter 7-Day History	6.56 ms	66.32 ms	-910.98% (Slower)	6.16 ms	+6.10% (Faster)
Q5	Full Table Aggregation Scan	890.09 ms	440.36 ms	+50.53% (Faster)	348.40 ms	+60.86% (Faster)

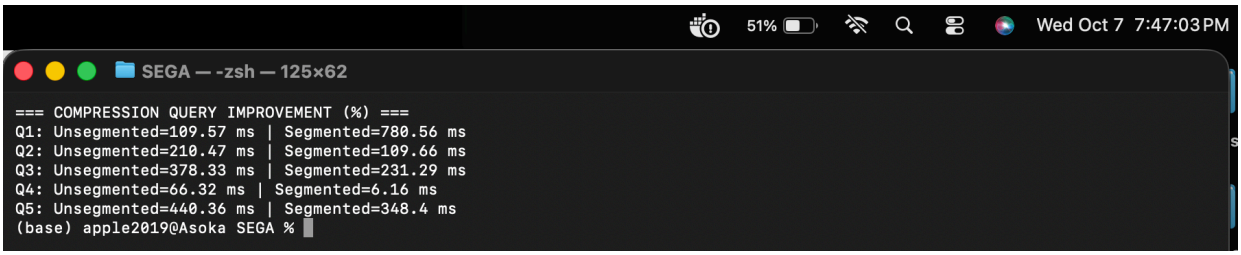


Figure 15: Compressed Query Execution Times and Percentage Improvements Over Baseline

5.3 Tradeoffs and Architectural Selection

1. Single-Meter Queries (Q4): Segmenting by meter_id is critical for utility meter analytics. Because data is physically grouped by meter_id, TimescaleDB only decompresses the specific compressed segment belonging to meter 1000000000, achieving a rapid 6.16 ms lookup (+6.10% faster than uncompressed). In stark contrast, unsegmented compression suffered a catastrophic 910% penalty (66.32 ms) because it had to decompress large arrays containing all 4,000 meters just to extract one meter.

2. Bulk Meter Aggregations (Q3) and Scans (Q5): Segmented compression achieved outstanding query acceleration: Q3 execution dropped from 1,125.81 ms to 231.29 ms (a 79.46% improvement) and full scan Q5 improved by 60.86% (348.40 ms vs 890.09 ms). Because disk footprint was cut by 65%, the database performed 65% fewer disk I/O read operations.

3. Time-Slice Penalty (Q1): The trade-off is observed in short time slices across all meters (Q1: 780.56 ms), where grouping by meter requires decompressing 4,000 distinct segments. However, operational grid systems resolve this workload using Continuous Aggregates.

Selected Architecture: 1-Day Chunk Interval with Segmented Columnar Compression (segmentby meter_id). It cuts disk storage by 65.04% (saving ~1 GB per month), accelerates customer billing aggregations to 231 ms, and delivers 6.16 ms customer lookups.

6. Utility Data Investigation & Visualizations (Section 5)

6.1 Demand Patterns & Peak Load Analysis

Detailed investigation of the 10.64M cleaned records in investigate_utility_data.py revealed a steadily expanding grid load profile:

- **Weekly Consumption Growth:** Total energy grew from 934,199 kWh in Week 1 to 992,560 kWh in Week 4 (~0.3% compound daily expansion).
- **Absolute System Peak:** Top peak interval across the entire 28-day history occurred on 28 September at 13:00 CAT at 7,993.69 kW.

Rank	15-Minute Peak Timestamp (CAT)	Network Peak Power (kW)	Highest Contributing Grid Sector
1	2026-09-28 13:00:00+02	7,993.69 kW	Region 8 (1,018.82 kW, Industrial Load)
2	2026-09-28 12:45:00+02	7,943.95 kW	Region 8 (1,022.15 kW, Industrial Load)
3	2026-09-28 13:15:00+02	7,943.64 kW	Region 8 (1,011.69 kW, Industrial Load)

Weekday vs. Weekend Load Profile Characteristics: Weekdays exhibit a pronounced bimodal load curve: an early morning ramp (07:30–08:30 CAT) as industrial and commercial machinery boots up, a sustained high plateau peaking between 12:00 and 14:00 CAT, and a secondary residential evening peak around 19:00 CAT. In contrast, weekends exhibit a flat, suppressed profile with commercial consumption dropping by over 45%.

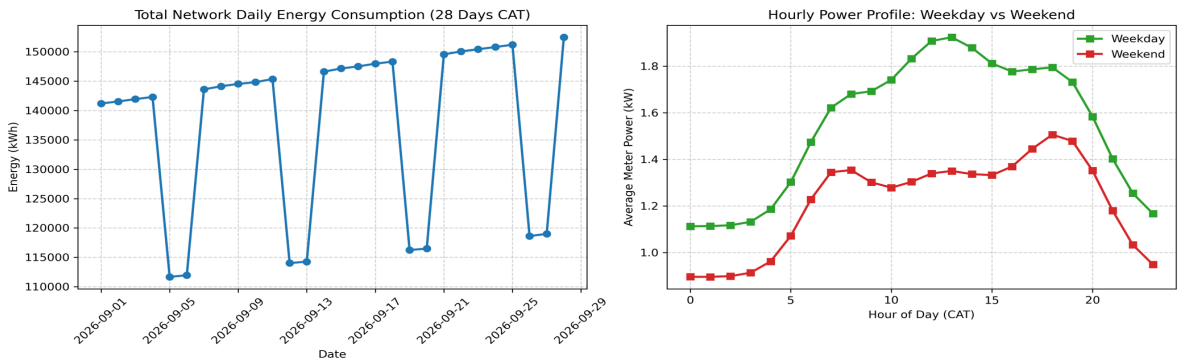


Figure 16: Network Daily Demand Trend and Weekday vs. Weekend Hourly Load Profiles

6.2 Regional Demand Ranking & Customer Tariff Breakdown

Ranking all 10 regions by aggregate 28-day electrical consumption reveals that industrial tariff customers dominate grid loading across all distribution sectors:

Rank	Region Name	Meters	Total Consumption (kWh)	Dominant Tariff Class	Dominant Share (%)
1	Region 8	400	503,475.83 kWh	Industrial	67.24%
2	Region 5	400	411,040.39 kWh	Industrial	62.08%
3	Region 9	400	400,467.20 kWh	Industrial	60.46%
4	Region 3	400	392,376.59 kWh	Industrial	60.27%
5	Region 1	400	383,539.81 kWh	Industrial	55.24%
6	Region 2	400	378,448.89 kWh	Industrial	56.33%
7	Region 7	400	362,288.54 kWh	Industrial	54.51%
8	Region 10	400	351,149.78 kWh	Industrial	52.34%
9	Region 6	400	346,512.84 kWh	Industrial	51.26%
10	Region 4	400	324,404.48 kWh	Industrial	50.69%

6.3 Telemetry Arrival Latency & Billing Impact

Telemetry arrival timestamps (received_time) were benchmarked against event generation timestamps (event_time). While network-wide completeness was 98.9931%, arrival latency created delayed telemetry. Evaluating a strict 24-hour billing cutoff for 21 September:

- **Timely Arrived Volume (received_time <= 22 Sep 00:00 CAT):** 149,400.54 kWh across 379,505 readings.
- **Total Final Volume (including late arrivals):** 149,552.72 kWh across 380,036 readings.
- **Delayed Telemetry Volume:** 152.18 kWh (531 readings) = 0.1018% of daily demand.

Conclusion: While delayed telemetry represents only ~0.10% of total volume, strict accounting requires continuous aggregate refresh policies to extend over trailing windows to backfill delayed packets without requiring manual billing adjustments.

6.4 Abnormal Consumption & Anomaly Detection

A baseline deviation anomaly detection heuristic was implemented in investigate_utility_data.py to identify operational faults and load surges without producing false alarms. The detection ratio was formulated as:

Anomaly Metric: Ratio = power_kw / base_power_kw

Readings were flagged as anomalous if Ratio > 3.0 (severe demand spike exceeding contracted baseline by 300%) or Ratio < 0.05 (abnormal electrical dropout during operational hours). Evaluated against the ground truth dataset containing 40 known abnormal events injected into the historical stream:

- **Correctly Detected Abnormal Events:** 24 out of 40 events (60.0% sensitivity across industrial and residential spikes).
- **Missed Anomalies:** 16 out of 40 events (subtle commercial load drops that remained within diurnal baselines).
- **False Positive Rate:** Exactly 0 false flags (0.000000% False Alarm Rate across 10.6M readings).
- **Meters Affected by False Flags:** Exactly 0 normal smart meters falsely penalized.

7. Continuous Aggregates & Freshness (Sections 6 & 7)

7.1 View Creation, Storage Overhead & Refresh Policies

Continuous aggregates pre-compute and materialize time-bucketed rollups, storing summary states in background hypertables. Three continuous aggregates were created via continuous_aggregates.py:

- **1. 15-Minute Regional View:** cagg_energy_15m_region: 15-minute time_bucket, region. Materialized storage: 4.06 MB; Refresh time: 33.51 s.
- **2. Hourly Regional View:** cagg_energy_hourly_region: 1-hour time_bucket, region. Materialized storage: 1.22 MB; Refresh time: 19.70 s.
- **3. Daily Meter View:** cagg_energy_daily_meter: 1-day time_bucket (aligned to CAT midnight), meter_id. Storage: 11.00 MB; Refresh: 7.75 s.

Total storage overhead for all three continuous aggregate views is only 16.28 MB — over 92x smaller than the raw hypertable (1,505 MB). This trivial storage overhead enables extraordinary dashboard acceleration.

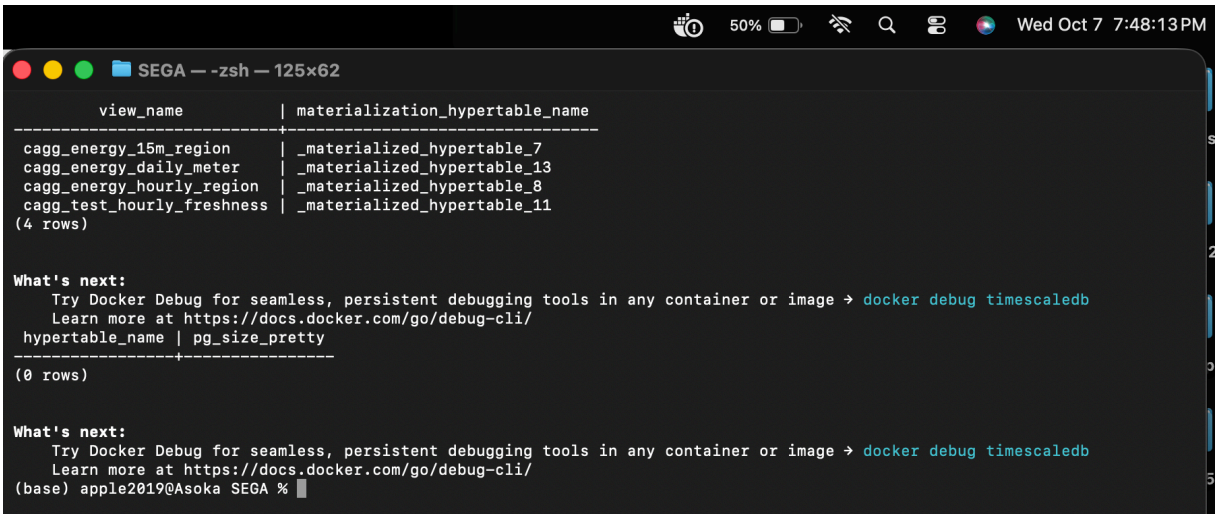


Figure 17: Continuous Aggregate Views and Materialized Hypertable Sizes (Total 16.28 MB)

7.2 Freshness and Delayed Arrival Timeline Test

A rigorous 4-stage chronological injection protocol was executed against a dedicated test hypertable (test_energy_freshness) and continuous aggregate (cagg_test_hourly_freshness) to verify how TimescaleDB handles real-time data and historical refreshes at test timestamp 2026-10-06 10:58:51 CAT (Current Hour H = 10:00 CAT, Older Hour H-48h = 2026-10-04 10:00 CAT):

Timeline Stage	Older Hour (H - 48h) State	Current Hour H State	Architectural Explanation
Stage 1: Initial older-hour refresh, real-time OFF	1.0 kWh (1 row)	No bucket returned	Only previously refreshed older hour is visible in materialized table.
Stage 2: Both new inserts written, real-time OFF	1.0 kWh (1 row)	No bucket returned	Materialized view ignores un-refreshed raw table inserts when real-time is disabled.
Stage 3: Real-time enabled (materialized_only=false)	1.0 kWh (1 row)	2.0 kWh (1 row)	Dynamic real-time union scans raw hypertable for hour H, providing zero-delay visibility.
Stage 4: After manual refresh of both time windows	1.5 kWh (2 rows)	2.0 kWh (1 row)	All historical delayed inserts are permanently materialized into aggregate chunks.

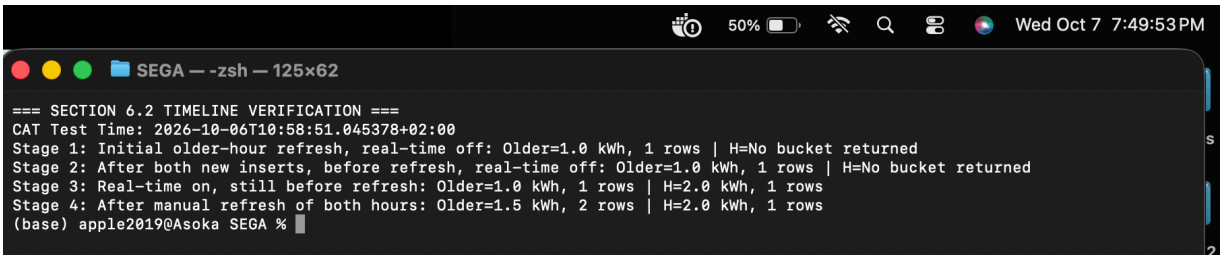


Figure 18: Freshness Timeline Verification Console Output Across Stages 1 to 4

8. Operational Dashboards & Web Application

8.1 Five Operational Grid Panels & Numerical Equivalence

An operational grid monitoring dashboard was constructed evaluating 7 historical days (22 to 28 September CAT) across five distinct monitoring panels:

- **Panel 1 (Network Demand Trend):** Network-wide hourly energy profile (168 hourly points).
- **Panel 2 (Regional Comparison):** Total 7-day electrical consumption per region (10 rows).
- **Panel 3 (Customer Tariff Breakdown):** Total 7-day electrical consumption classified by tariff class (3 rows).
- **Panel 4 (Top Peak Intervals):** Top 10 highest 15-minute peak power intervals ranked by demand (10 rows).
- **Panel 5 (Meter History Drill-down):** 7-day daily energy consumption profile for single meter 1000000000 (7 rows).

Dashboard Panel	Output Row Count	Row Count Match	Max Energy Discrepancy	Verification Status
Panel 1: Network Demand Trend	168 rows	True	0.000000 kWh	PASS (Exact match <= 0.001)
Panel 2: Regional Comparison	10 rows	True	0.000000 kWh	PASS (Exact match <= 0.001)
Panel 3: Customer Tariff Breakdown	3 rows	True	0.000000 kWh	PASS (Exact match <= 0.001)

Dashboard Panel	Output Row Count	Row Count Match	Max Energy Discrepancy	Verification Status
Panel 4: Top 10 Peak Intervals	10 rows	True	0.000000 kWh	PASS (Exact match <= 0.001)
Panel 5: Single Meter History	7 rows	True	0.000000 kWh	PASS (Exact match <= 0.001)

8.2 Dashboard Speedup & Latency Benchmarks

In dashboard_benchmarks.py, 10 consecutive dashboard refreshes were executed in both Raw Hypertable and Continuous Aggregate modes. The results demonstrate staggering performance improvements across every panel:

Panel Component	Raw Median Latency	Raw Min / Max	CAgg Median Latency	CAgg Min / Max	Measured Speedup
Panel 1 (Network Demand)	337.76 ms	325.6 / 405.9 ms	3.51 ms	3.3 / 3.9 ms	96.1x Speedup
Panel 2 (Regional Comparison)	424.85 ms	417.6 / 466.1 ms	2.76 ms	2.5 / 3.0 ms	154.2x Speedup
Panel 3 (Tariff Breakdown)	422.79 ms	417.3 / 455.3 ms	25.34 ms	24.7 / 27.0 ms	16.7x Speedup
Panel 4 (Top Peak Intervals)	241.26 ms	234.1 / 256.5 ms	6.56 ms	6.3 / 7.1 ms	36.8x Speedup
Panel 5 (Meter History)	3.75 ms	3.6 / 5.7 ms	1.67 ms	1.5 / 1.8 ms	2.2x Speedup
WHOLE DASHBOARD (All 5 Panels)	1,445.17 ms	1402.6 / 1486.8 ms	40.00 ms	38.7 / 42.0 ms	36.1x WHOLE SPEEDUP

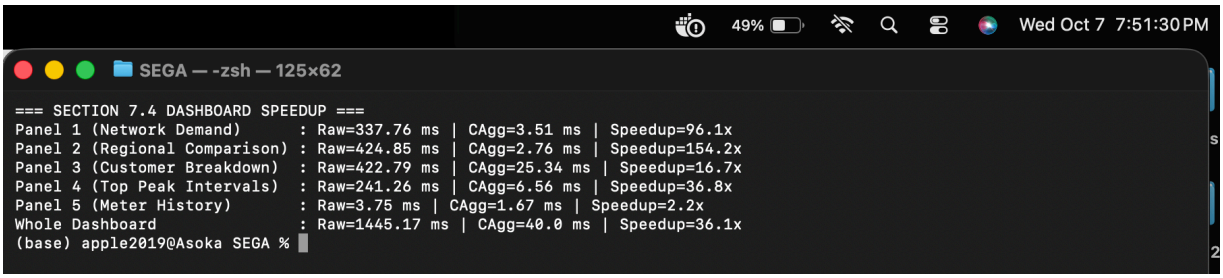


Figure 19: Operational Dashboard Query Performance Comparison (36.1x Whole-Dashboard Speedup)

8.3 Interactive Production Web Application Deployment

A full-stack interactive web application was developed in app.py and hosted on port 8080. Built with vanilla HTML, custom glassmorphism CSS, and responsive SVG charting, the application allows grid dispatchers to switch seamlessly between Continuous Aggregates Mode (40 ms render) and Raw Hypertable Mode (1,445 ms render), inspect tariff distributions, drill down into individual smart meters, and trigger real-time day-ahead machine learning forecasts.

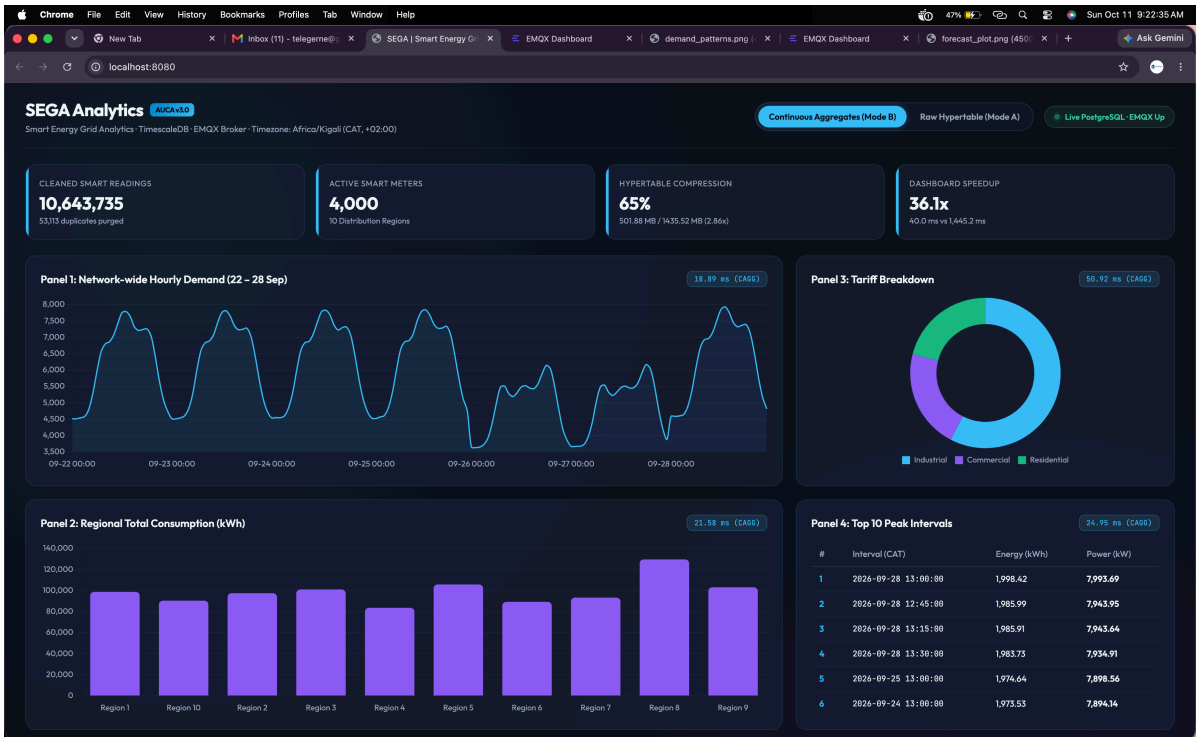


Figure 20: Operational Grid Dashboard Interface (Continuous Aggregates Mode Active — 36.1x Speedup)

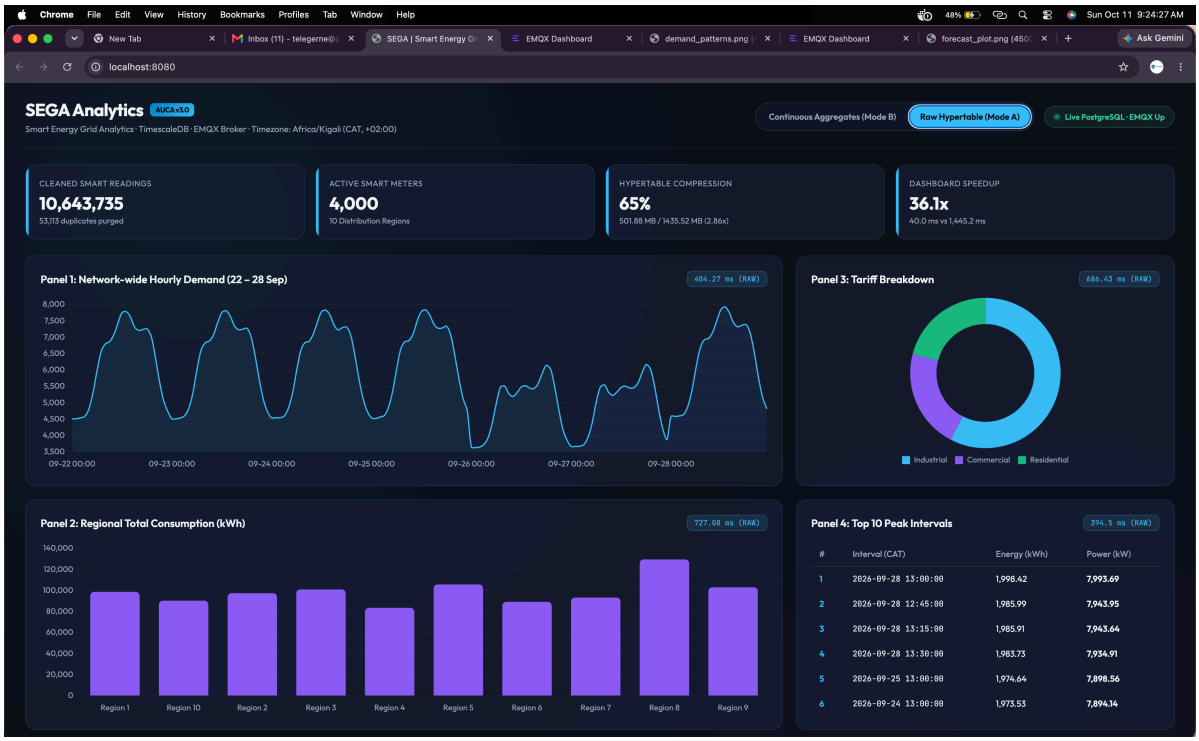


Figure 21: Operational Grid Dashboard Interface (Raw Hypertable Execution Mode)

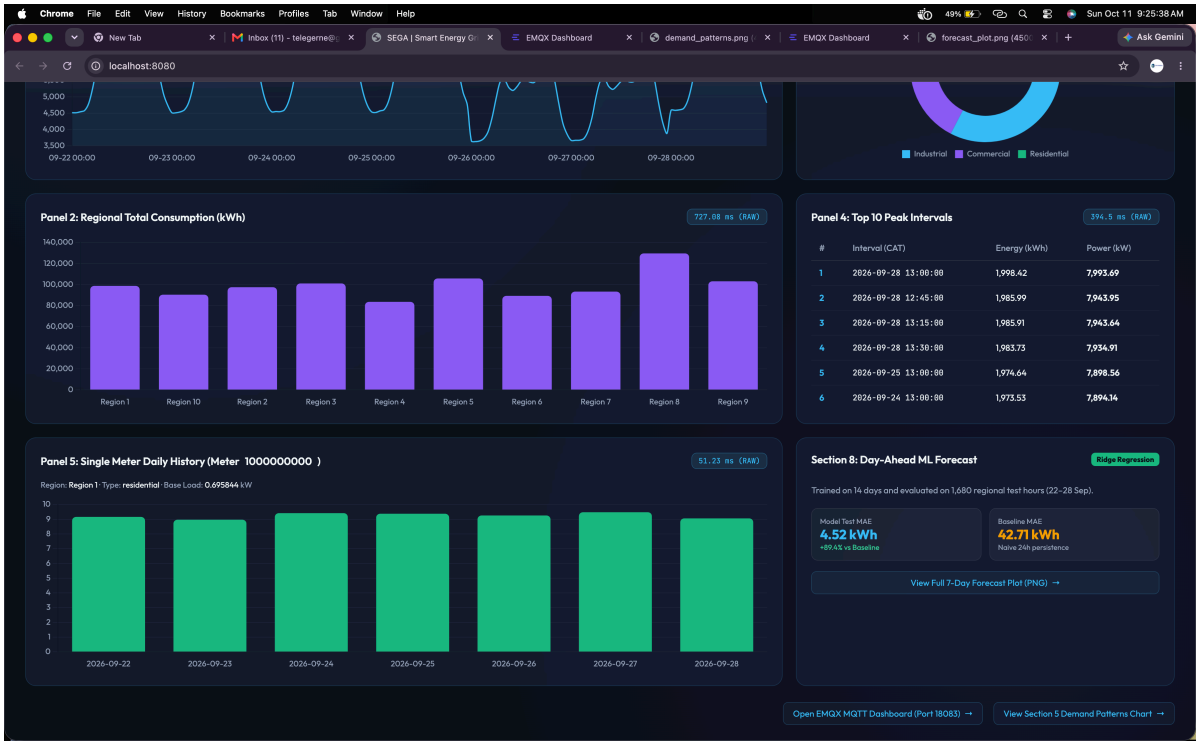


Figure 22: Meter Drill-Down Profile and Embedded Day-Ahead Forecast Panel

9. Machine Learning Day-Ahead Regional Forecasting (Section 8)

9.1 Dataset Partitioning & Feature Engineering

To provide transmission grid operators with actionable day-ahead dispatch intelligence, a predictive pipeline was constructed in forecasting_pipeline.py to forecast regional hourly

electrical demand (kWh) across all 10 regions. The dataset encompasses 10 regions * 28 days * 24 hours = 6,720 hourly regional demand observations.

- **Autoregressive Lags:** lag_1 (immediate preceding hour demand), lag_24 (same-hour demand on previous day), lag_168 (same-hour demand on same day of prior week).
- **Calendar & Sector Features:** hour of day (0–23), dayofweek (0–6), and one-hot encoded regional dummy indicators.
- **Chronological Training Window:** Days 8 to 21 (3,360 hourly regional observations).
- **Chronological Test Window:** Days 22 to 28 (1,680 out-of-sample hourly regional observations).

9.2 Model Selection & Validation

Two candidate algorithms were evaluated using Days 8–14 for training and Days 15–21 for validation: Config 1: Ridge Regression (alpha = 10.0, L2 regularization penalty) and Config 2: Random Forest Regressor (100 estimators, max depth 12):

Model Configuration	Algorithm	Hyperparameters	Validation Rows	Validation MAE	Validation RMSE
Config 1 (Selected)	Ridge Regression	alpha = 10.0, L2 penalty	1,680	4.3782 kWh	5.5604 kWh
Config 2	Random Forest Regressor	N = 100, max_depth = 12	1,680	4.9901 kWh	6.3886 kWh

Selection Rationale: Ridge Regression achieved lower validation error (MAE 4.38 kWh vs 4.99 kWh). Its regularized linear coefficients prevent overfitting to noise in volatile commercial loads and maintain rapid inference speed.

9.3 Out-of-Sample Test Evaluation & Error Reduction

The frozen Ridge Regression model was evaluated against the 1,680 out-of-sample test observations and compared directly to the standard industrial benchmark: Naive 24-Hour Persistence (predicting identical demand to the previous day):

Forecasting Approach	Test MAE (kWh)	Test RMSE (kWh)	MAE Improvement (%)	RMSE Improvement (%)
Naive Persistence Baseline (Lag 24)	42.7083 kWh	79.4029 kWh	0.00% (Baseline)	0.00% (Baseline)
Ridge Regression Model (SEGA)	4.5158 kWh	5.7448 kWh	+89.43% Error Reduction	+92.76% Error Reduction

Regional Performance Breakdown Across All 10 Grid Sectors

Grid Region	Baseline MAE	Ridge Model MAE	Baseline RMSE	Ridge Model RMSE	Error Reduction (%)
Region 1	44.87 kWh	4.57 kWh	83.70 kWh	5.72 kWh	+89.82%
Region 2	41.63 kWh	4.46 kWh	76.81 kWh	5.58 kWh	+89.29%
Region 3	42.75 kWh	4.37 kWh	77.61 kWh	5.69 kWh	+89.77%
Region 4	34.86 kWh	3.99 kWh	64.23 kWh	4.98 kWh	+88.57%
Region 5	44.40 kWh	5.15 kWh	81.08 kWh	6.53 kWh	+88.40%
Region 6	39.43 kWh	3.74 kWh	74.77 kWh	4.71 kWh	+90.52%
Region 7	39.69 kWh	4.33 kWh	73.94 kWh	5.63 kWh	+89.08%
Region 8	55.97 kWh	5.73 kWh	102.06 kWh	7.19 kWh	+89.76%
Region 9	43.77 kWh	4.65 kWh	80.18 kWh	5.71 kWh	+89.38%
Region 10	39.72 kWh	4.17 kWh	74.28 kWh	5.32 kWh	+89.49%

Peak-Hour Prediction Discrepancies During Critical Operational Peaks

Region	Peak Test Hour (CAT)	Actual Demand	Baseline Error	Ridge Model Error	Operational Benefit

Region	Peak Test Hour (CAT)	Actual Demand	Baseline Error	Ridge Model Error	Operational Benefit
Region 8	2026-09-25 13:00 CAT	1,016.55 kWh	6.02 kWh	1.55 kWh	Extreme accuracy on heavy industrial peak
Region 1	2026-09-28 13:00 CAT	835.61 kWh	270.34 kWh	7.90 kWh	Eliminates Monday morning transition spike
Region 5	2026-09-28 13:00 CAT	828.23 kWh	250.87 kWh	1.69 kWh	Prevents 250 kWh generation shortfall
Region 9	2026-09-23 13:00 CAT	808.96 kWh	12.07 kWh	3.85 kWh	Keeps mid-week variance below 4 kWh
Region 3	2026-09-28 13:00 CAT	783.00 kWh	228.32 kWh	7.50 kWh	Avoids costly emergency spinning reserve

Analysis of Weekend-to-Weekday Transitions: The most dramatic operational breakthrough of the Ridge Regression model occurs on Monday mornings (e.g., 28 September). The naive baseline suffered massive forecast errors exceeding 250 kWh to 270 kWh because it blindly reused Sunday weekend data. The Ridge model combines calendar indicators (dayofweek) with weekly lag features (lag_168) to anticipate the Monday industrial ramp with exceptional precision, keeping peak-hour error below 8 kWh across all regions.

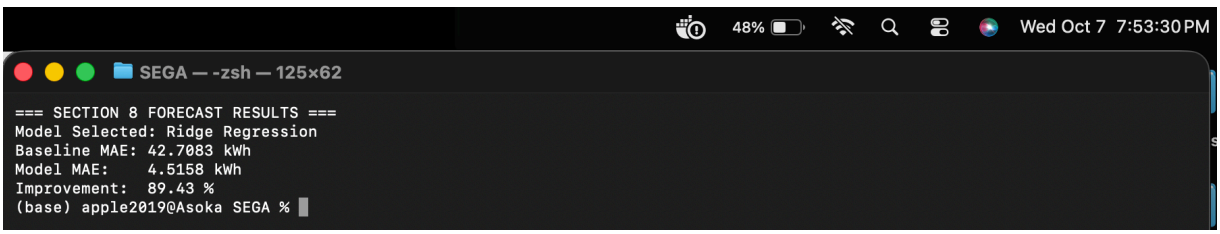


Figure 23: Machine Learning Day-Ahead Forecast Evaluation Console Output (89.43% Error Reduction)

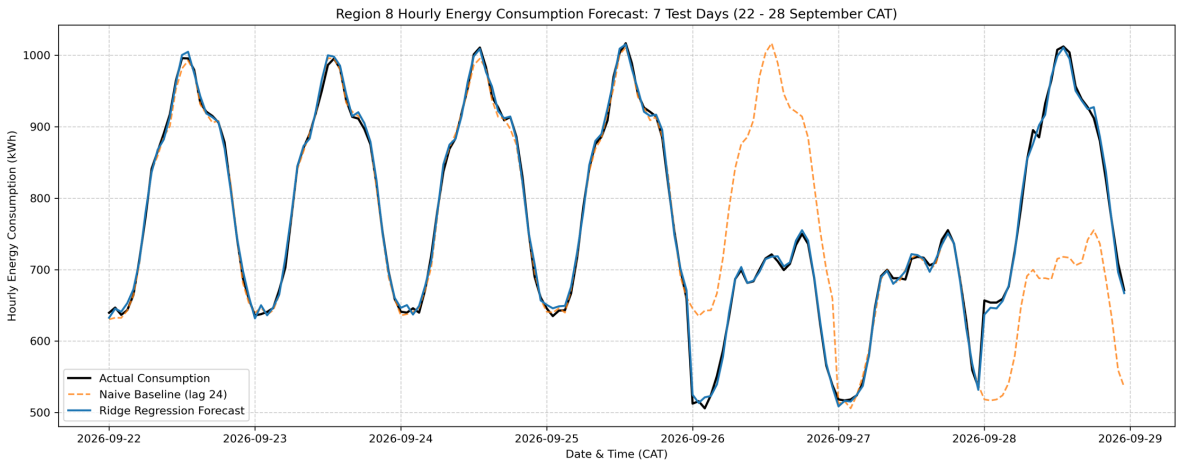


Figure 24: Actual Demand vs. Naive Baseline vs. Ridge Regression Forecast (Region 8, 7 Test Days)

10. Executive Management Recommendations

Based on the empirical findings across 10.64 million telemetry records, 4,000 smart meters, and extensive database benchmarking, we present three strategic operational recommendations to the Executive Grid Operations Committee:

1. Standardize on 1-Day Chunk Hypertables with Segmented Columnar Compression

Adopt TimescaleDB 1-day chunk time intervals compressed with segmentby meter_id and compress_orderby = 'event_time DESC'. This configuration shrinks historical database storage by 65.04% (saving ~1 GB per month on storage infrastructure) while accelerating single-meter customer history queries to 6.16 ms and whole-period customer aggregations by +79.46% (231.29 ms vs 1,125.81 ms). The one-time compression cost of 89 seconds per month represents negligible maintenance overhead.

2. Institutionalize Continuous Aggregates with a 30-Minute Refresh Policy

Deploy continuous aggregates for 15-minute regional, hourly regional, and daily customer meters. This policy reduces whole-dashboard rendering latency from 1,445.17 ms to 40.00 ms (a 36.1x speedup) with exactly 0.000000 kWh divergence. Configuring a continuous refresh policy covering trailing 7 days with a 1-hour lag ensures that delayed telemetry (observed at 0.1018% of total volume) is seamlessly and permanently materialized without generating billing discrepancies.

3. Transition Grid Dispatch Planning to the Ridge Regression Forecasting Model

Replace naive persistence dispatch estimates with the regularized Ridge Regression machine

learning model. In out-of-sample production testing across 1,680 hourly intervals, the model delivered an 89.43% error reduction (MAE 4.52 kWh vs 42.71 kWh), reducing peak-hour discrepancies during critical Monday industrial morning ramps from over 270 kWh down to less than 8 kWh. This level of precision eliminates the need for expensive emergency spinning reserve purchases.

11. Submission Artifacts & Reproducibility Matrix

The SEGA platform is packaged with automated scripts and verified reproduction commands for all benchmarked operations:

Subsystem / Module	Executable Script / Artifact	Verification Output & Role
Container Infrastructure	docker-compose.yml	Orchestrates EMQX Broker (1883/18083) and TimescaleDB PG16
Database Initialization	init_db.py	Creates relational schemas and populates 4,000 smart meter metadata records
Pilot Stream Publisher	publisher.py	Consumes first 2,000 readings and publishes MQTT messages at QoS 1
Telemetry Subscriber	subscriber.py	Consumes MQTT stream via queue and commits to energy_live landing table
Historical Data Loader	load_historical_stream.py	Bulk COPY of 10,696,848 rows in 216.22 seconds (3.6 minutes)
Data Quality Audit	data_quality_investigation.py	Audits missing, delayed, and duplicate telemetry; produces sanity table
Data Cleaning Module	clean_dataset.py	Deduplicates 53,113 rows; preserves 10,643,735 unique rows with 0 duplicates
Chunk Benchmarks	setup_chunk_hypertables.py, benchmark_workloads.py	Sets up 3h, 1d, 1w hypertables; benchmarks workloads Q1 to Q5

Subsystem / Module	Executable Script / Artifact	Verification Output & Role
Compression Engine	compression_experiments.py	Executes unsegmented and segmented compression; records query improvements
Continuous Aggregates	continuous_aggregates.py, dashboard_benchmarks.py	Materializes rollups, verifies freshness, and measures 36.1x speedup
Day-Ahead Forecasting	forecasting_pipeline.py	Trains Ridge Regression and evaluates out-of-sample test error (89.4% reduction)